

Tool Registry – Proof of Concept Documentation

1. What We Are Building

We are building a **Tool Registry** for an LLM-based system using **Python, FastAPI, LangChain, and LangGraph**.

The registry acts as a **central catalog of tools** that an agent can reference when deciding which tool to use. This PoC focuses only on **tool definition and management**, not execution.

Tool Types in Scope

1. Prebuilt Tools

2. LangChain built-in tools
3. Developer-defined tools
4. Stored as references (not executable code)

5. Custom API Tools

6. User-defined HTTP tools
7. Supports **GET / POST** APIs
8. User provides:
 - API URL
 - Method
 - Input schema
 - Output schema
 - Custom message for LLM guidance

Custom function tools and advanced execution logic are **explicitly out of scope** for this PoC.

2. Core Design Principles

- **Single-user system**
- **JSON file as persistence layer** (no database)
- **JSON Schema** for tool inputs and outputs
- **No authentication, authorization, or tool execution logic**
- **Custom message is used only to guide the LLM's output selection**

Each tool definition is static metadata that can later be consumed by LangChain / LangGraph.

3. How We Are Doing It

3.1 Tool Registry Storage

All tools are stored in a single JSON file:

```
app/registry/tool_registry.json
```

This file contains:

- Prebuilt tools
- Custom API tools
- Tool schemas, configuration, and metadata

The JSON file acts as the **source of truth** for the registry.

3.2 Tool Definition Model

Each tool follows a common structure:

- `id` - unique identifier
- `type` - `prebuilt` or `custom_api`
- `name` - tool name exposed to the LLM
- `description` - used by the LLM for tool selection
- `input_schema` - JSON Schema defining tool inputs
- `output_schema` - JSON Schema defining tool outputs
- `api_url` - target API endpoint (for custom API tools)
- `api_request_type` - HTTP method (GET or POST)
- `custom_message` - instructions for the LLM to extract only required fields from API responses
- `metadata` - creation info

For **custom API tools**, the `config` includes:

- API URL
 - HTTP method (GET/POST)
 - Query parameters or request body template
-

3.3 Application Structure

```
app/  
├─ main.py  
├─ registry/  
│   └─ tool_registry.json
```

```
|   |— loader.py
|   |— manager.py
|   |— schemas.py
|— api/
|   |— tools.py
|— utils/
|   |— json_store.py
```

3.4 Responsibilities by Layer

API Layer (`api/tools.py`)

- Exposes HTTP endpoints
- Accepts user input
- Returns tool definitions

Endpoints:

- `GET /tools`
- `GET /tools/{id}`
- `POST /tools`
- `DELETE /tools/{id}`

Registry Layer (`registry/`)

- ``
 - Pydantic models for tool validation
 - Ensures correct JSON Schema and config structure
 - ``
 - Loads tools from JSON at application startup
 - ``
 - In-memory tool registry
 - Add, fetch, delete tools
 - Enforces unique IDs and tool types
-

Persistence Layer (`utils/json_store.py`)

- Reads from `tool_registry.json`
 - Writes updates back to disk
 - Isolated from business logic
-

4. System Flow

4.1 Application Startup

1. FastAPI starts
 2. Tool registry JSON is loaded
 3. Tools are stored in memory via `ToolManager`
-

4.2 Adding a Custom API Tool

1. User sends `POST /tools`
 2. Input is validated using Pydantic schemas
 3. Tool is added to in-memory registry
 4. Registry is persisted to JSON
-

4.3 Listing or Fetching Tools

1. User calls `GET /tools` or `GET /tools/{id}`
 2. Tools are returned from in-memory registry
-

5. Outcome

This PoC delivers:

- A clean, extensible tool registry
- JSON-based persistence
- LangChain-compatible tool metadata
- Minimal complexity with clear upgrade paths

The design can later evolve to support databases, auth, execution engines, and multi-user setups without breaking the core structure.